Backend-ul, feature cu feature

Spring Boot 4.1 · Java 21 · Maven · MySQL 8 · Docker · GitHub Actions · EC2 · Rareș

Ținta e în `booking-docs/`: `STRUCTURA_BD.pdf` = tabelele, `FEATURES.pdf` = ce trebuie să meargă, `openapi.yaml` = contractul (îl deschizi în `editor.swagger.io`; DTO-urile = schemele, metodele din controllere = `operationId`).
Fiecare feature trece prin același drum:
migrare `Vn__…sql` (dacă schimbi tabele) → repository → service → controller → test de integrare → push → verde în Actions → verificat pe EC2

1 Frizeri — scheletul, cu tot drumul până pe EC2

a) Fundația proiectului

- Întâi să compileze: `@Builder` de pe `Customer` cere un constructor cu toate câmpurile, pe care Lombok nu-l mai face când ai scris tu constructori. Scoate-l.
- `<java.version>21</java.version>`; `git update-index --chmod=+x booking-api/mvnw`, altfel `./mvnw` nu pornește în Actions.
- `spring-boot-starter-flyway` + `flyway-mysql`; `spring.jpa.hibernate.ddl-auto: validate`. Conexiunea din variabile de mediu: `DB_URL`, `DB_USER`, `DB_PASSWORD`, cu valori implicite pentru local.
- Testele pe MySQL din Testcontainers: `spring-boot-testcontainers`, `@ServiceConnection`.

b) Schema întreagă, o singură dată

- Entitățile le ai deja pe toate, iar `validate` le verifică pe toate la pornire. De aceea `V1__schema.sql` creează **toate cele 9 tabele** din `STRUCTURA_BD.pdf`, cu constrângerile lor. Pornește pe o bază goală = gata.
- Aliniezi entitățile la PDF: `User` + `Role` în pachetul `users`; `user_id` în `Customer` și `Barber`, fără `email`; `barber` în `Procedure` și în `RecurringAppointment`; `uniqueConstraints` noi.

c) Codul

- Pachetul `barbers`: `BarberQueryService` și `BarberCommandService` cu implementări, `BarberController`; DTO-uri `BarberCreateRequest` / `BarberUpdateRequest` / `BarberResponse` ca `record`.
- `createBarber` salvează `User` (rol `BARBER`) + `Barber` într-o tranzacție. Parola o treci prin `BCryptPasswordEncoder` din `spring-security-crypto` — doar atât; starter-ul de securitate vine la 3, altfel blochează tot.
- Cele 5 operații din F1 exact ca în `openapi.yaml`: câmp gol → 400, email existent → 409, id inexistent → 404.
- Erorile într-un singur `@RestControllerAdvice`, ca `ProblemDetail`, cu câte o metodă pe tip de răspuns (404, 409, 400) — îl refolosești la toate feature-urile.
- Un test de integrare (`@SpringBootTest` + MockMvc): adaugi → apare în listă; email duplicat → 409; nume gol → 400; retras → nu mai apare.

d) Docker, local

- `booking-api/Dockerfile` în două etape: build cu Maven → rulare pe `eclipse-temurin:21-jre`.
- `booking-ops/docker-compose.yml`: `mysql` (volum + `healthcheck`) și `api` (`depends_on` cu `condition: service_healthy`, port 8080).
- Parolele vin din `booking-ops/.env`, care e în `.gitignore`; în repo pui doar `.env.example`.

> **Gata când:** `docker compose up --build` → `GET http://localhost:8080/api/barbers` = 200.

e) GitHub Actions

- Fișierul stă în `.github/workflows/api.yml`, **la rădăcina repo-ului** (nu în `booking-ops/`). Pornește la push pe `master` când se schimbă `booking-api/**` sau `booking-ops/**`.
- Job `test`: `./mvnw verify` (runner-ul `ubuntu-latest` are Docker, deci Testcontainers merge).
- Job `image`: imaginea în GHCR, `ghcr.io/raresdmt-boop/booking-api`, cu tag-urile `latest` și SHA-ul commit-ului; platforma = arhitectura instanței EC2 (`amd64` sau `arm64`). Pachetul îl faci public, ca EC2 să-l poată trage fără login.
- Job `deploy`: `scp` cu `docker-compose.yml` pe EC2, apoi prin `ssh`: `docker compose pull && docker compose up -d`. Secrete în repo: `EC2_HOST`, `EC2_USER`, `EC2_SSH_KEY`.

f) EC2

- Ubuntu, Docker + pluginul compose. Security Group: 22 doar de la IP-ul tău, 8080 deschis (temporar, până la nginx).
- Folderul `~/booking/` cu `.env` pus o singură dată, de mână. `.env` nu trece niciodată prin repo.

> **Gata când:** un push pe `master` → workflow verde → `curl http://<ip-ec2>:8080/api/barbers` = 200.

2 Procedurile frizerului

- Sub-resursă a frizerului: `/api/barbers/{id}/procedures`. `getBarberProcedures` (doar active), `createBarberProcedure`, `updateBarberProcedure`, `deactivateBarberProcedure` (`active = false`).
- Procedura o cauți după **ambele** id-uri: `findByIdAndBarberId` — altfel `PUT /api/barbers/1/procedures/7` modifică procedura lui 2.

> **Gata când:** doi frizeri au amândoi „Tuns”, cu prețuri diferite, iar al doilea „Tuns” la același frizer = 409.

3 Conturi și clienți

- `register` salvează `User` + `Customer` în aceeași tranzacție; `login`, `getCurrentUser`, `getCustomers`, `setCustomerBlocked`.
- Securitate: `spring-boot-starter-security` + `spring-boot-starter-oauth2-resource-server`, JWT HS256 (`NimbusJwtEncoder`/`Decoder`), claim `role` → `ROLE_…`, `@EnableMethodSecurity`. Publice: `register`, `login`, GET-urile din contract, Swagger UI, `/error`.
- Un `ApplicationRunner` creează contul `ADMIN` din `ADMIN_EMAIL` / `ADMIN_PASSWORD` dacă nu există. Acestea și `JWT_SECRET` stau în `.env` pe EC2 și ca secrete în Actions — niciodată în repo.
- Frizerii devin doar pentru `ADMIN`; procedurile, pentru `ADMIN` sau frizerul însuși (403). `springdoc-openapi-starter-webmvc-ui` **3.1.x** (2.x nu merge pe Boot 4).

> **Gata când:** pe EC2: `register` → `login` → `/api/users/me` cu `Bearer` = 200; fără token = 401; `POST /api/barbers` fără ADMIN = 403; frizerul creat la 1 se loghează cu rolul `BARBER`.

4 Program și concedii

- `setBarberSchedule` (șterge programul vechi și îl salvează pe cel nou, într-o tranzacție), `getBarberSchedule`, `createBarberTimeOff`, `getBarberTimeOffs`, `deleteBarberTimeOff`.
- Cine are voie: `ADMIN`, sau frizerul al cărui `user_id` e în token — se verifică în service (403).

> **Gata când:** testul „frizerul A își schimbă programul — 200; programul lui B — 403” trece în Actions.

5 Ore libere

- `getAvailability`: `barberId`, `date`, `procedureIds` (ale lui, altfel 400) → listă de ore. E doar un calcul, fără migrare. Din programul zilei scazi perioadele libere și programările ne-anulate, apoi încerci fiecare oră din 15 în 15 minute.
- Calculul stă într-o clasă fără Spring, cu teste unitare pe cazuri mici: zi liberă, programare la mijlocul zilei, durată care nu mai încape la final.

> **Gata când:** pe EC2, o zi cu o programare la 10:00 nu mai propune 10:00 pentru o procedură de 30 de minute.

6 Programări

- `createAppointment` refolosește calculul de la 5 și refuză ora dacă nu e liberă (409); prețul și durata le copiază din procedurile frizerului. Doi clienți în aceeași secundă: citești frizerul cu `@Lock(PESSIMISTIC_WRITE)` la începutul tranzacției, așa că a doua cerere așteaptă și vede programarea primei.
- `getMyAppointments`, `getBarberAppointments`, `getAppointment` (procedurile cu `@EntityGraph`), `cancelAppointment`, `updateAppointmentStatus`.
- Testul de integrare al întregului flux, **prin login real** (nu doar `@WithMockUser`): programare (201) → aceeași oră din nou (409) → anulare (204) → aceeași oră (201).

> **Gata când:** fluxul complet merge din Swagger UI-ul de pe EC2.

7 Programări recurente

- `createRecurringAppointment` generează datele seriei, verifică fiecare dată cu calculul de la 5 și salvează seria + toate programările într-o singură tranzacție; o dată ocupată → 409 cu lista datelor, nimic salvat.
- `getMyRecurringAppointments`, `cancelRecurringAppointment` (`active = false` + anulează programările viitoare).

> **Gata când:** Swagger UI-ul de pe EC2 are aceleași 29 de operații ca `openapi.yaml`, și toate 7 feature-urile sunt bifate în `FEATURES.pdf`.

Spring Boot 4 față de exemplele de pe internet (scrise pe Boot 3): `@MockBean` → `@MockitoBean` · `@WebMvcTest` din `org.springframework.boot.webmvc.test.autoconfigure` · `@DataJpaTest` din `org.springframework.boot.data.jpa.test.autoconfigure` · Jackson 3: `tools.jackson.databind.ObjectMapper` · starter-ele de test sunt separate: `…-webmvc-test`, `…-data-jpa-test`, `…-validation-test`.